INSTRUKCJA PRAKTYCZNA • INF.04

Menedżer haseł w React

Budujemy aplikację krok po kroku: od dostarczonego szablonu HTML do działającego formularza i podsumowania danych.

Cel: po kliknięciu „Zapisz” aplikacja wyświetla adres, login oraz hasło albo komunikat „Hasło automatycznie generowane”. Checkbox decyduje, czy pole hasła jest aktywne.

Materiały i przygotowanie

Pracuj w nowym projekcie „Aplikacja React” (JavaScript) w Visual Studio. Potrzebujesz pliku szablon.txt z materiałów egzaminacyjnych oraz działających zależności projektu React i Bootstrap.

Ważne: jeżeli na stanowisku nie ma internetu, zależności npm i Bootstrap muszą być wcześniej zainstalowane. Sam kod aplikacji uruchomisz lokalnie.

Jak korzystać z instrukcji?

Wykonuj kroki po kolei. W miejscach oznaczonych „SPRAWDŹ” uruchom aplikację i potwierdź efekt. Nie wklejaj kolejnego fragmentu, jeśli poprzedni nie działa.

01 Pusty komponent App

W pliku src/App.jsx usuń kod przykładowy i wpisz:

```
function App() {
    return (
        <div>
            <h1>Menedżer haseł - wykonał 00000000000</h1>
        </div>
    );
}

export default App;
```

Zamiast zer wpisz swój numer zgodnie z poleceniem nauczyciela. Fragment wewnątrz return opisuje elementy, które pojawią się na stronie.

SPRAWDŹ: Na stronie widać nagłówek. Nie ma jeszcze formularza.

02 Wykorzystaj szablon otrzymany na egzaminie

Otwórz szablon.txt. Zawiera formularz HTML: dwa pola oraz przycisk. Skopiuj zawartość od <form> do </form> pod nagłówkiem w głównym div.

Szablon jest napisany w HTML. React używa JSX, więc w skopiowanym kodzie trzeba zmienić trzy rzeczy:

```
class="form-control"   ->  className="form-control"
for="pole"            ->  htmlFor="pole"
<input ...>            ->  <input ... />
```

Pamiętaj o wszystkich miejscach występowania class i for. Zmieniaj znaczniki input na samozamykające. Nie kasuj klas Bootstrap: pochodzą z szablonu.

03 Dostosuj pola do arkusza

Pierwsze pole szablonu dotyczyło e-maila. Zamień je na Adres WWW. Zastąp pierwszy blok form-group poniższym:

```
<div className="form-group m-4">
    <label htmlFor="adres">Adres WWW:</label>
    <input
        type="text"
        className="form-control"
        id="adres"
        placeholder="Adres strony WWW"
    />
</div>
```

Drugie pole formularza zmień na Login (zmieniamy także typ z password na text):

```
<div className="form-group m-4">
    <label htmlFor="login">Login:</label>
    <input
        type="text"
        className="form-control"
        id="login"
        placeholder="Twój login"
    />
</div>
```

SPRAWDŹ: Możesz wpisać adres WWW i login. W obu polach widać odpowiednie etykiety i podpowiedzi.

04 Dodaj trzecie pole: Hasło

Skopiuj jeden blok form-group, wklej go pod polem Login i dostosuj. W tym polu znaki mają być ukrywane.

```
<div className="form-group m-4">
    <label htmlFor="haslo">Hasło:</label>
    <input
        type="password"
        className="form-control"
        id="haslo"
    />
</div>
```

Dlaczego type="password"? Przeglądarka ukrywa wpisane znaki. To nie jest szyfrowanie, tylko sposób wyświetlania.

05 Dodaj checkbox, na razie bez funkcji

Wstaw ten fragment przed blokiem z polem Hasło:

```
<div className="form-group m-4">
    <input type="checkbox" id="generuj" defaultChecked />
    <label htmlFor="generuj"> Wygeneruj hasło</label>
</div>
```

defaultChecked zaznacza pole przy pierwszym wyświetleniu. Później zamienimy je na stan sterowany przez React.

06 Zmień przycisk szablonu

W istniejącym przycisku zamień tekst „Submit” na „Zapisz” i uzupełnij niebieski styl Bootstrap:

```
<button type="submit" className="btn btn-primary m-2">
    Zapisz
</button>
```

Jeżeli Bootstrap jest zainstalowany, zaimportuj jego CSS na górze App.jsx:

```
import "bootstrap/dist/css/bootstrap.min.css";
```

SPRAWDŹ: Widać 3 pola tekstowe, zaznaczony checkbox i niebieski przycisk. Kliknięcie „Zapisz” może na razie przeładować stronę - naprawimy to później.

07 Jak uzależnić pole hasła od checkboxa?

Postaw problem: gdy checkbox jest zaznaczony, pole hasła powinno być nieaktywne. Najpierw sprawdź, co robi HTML-owy atrybut disabled.

Do pola Hasło dopisz na chwilę disabled. Spróbuj coś wpisać. Teraz pole jest stale nieaktywne. Potrzebujemy stanu, który będzie się zmieniał.

Na samej górze pliku dopisz (obok importu Bootstrapa):

```
import { useState } from "react";
```

Wewnątrz App(), przed return, dodaj:

```
const [generujHaslo, setGenerujHaslo] = useState(true);
```

Znaczenie: generujHaslo to obecna wartość true/false; setGenerujHaslo zmienia ją; true jest wartością początkową.

08 Łączymy checkbox ze stanem

Przed return dodaj funkcję, która odwraca wartość przy każdej zmianie zaznaczenia:

```
function zmienGenerowanie() {
    setGenerujHaslo(!generujHaslo);
}
```

W checkboxie usuń defaultChecked i wstaw:

```
checked={generujHaslo}
onChange={zmienGenerowanie}
```

W polu Hasło zamiast stałego disabled wpisz:

```
disabled={generujHaslo}
```

Jak to działa? Checkbox zmienia generujHaslo między true i false. Pole Hasło otrzymuje disabled=true (zablokowane) lub disabled=false (aktywne).

SPRAWDŹ: Zaznaczony checkbox blokuje wpisywanie hasła; odznaczony je umożliwia.

09 Odczytaj tekst z pola Adres WWW

Użytkownik wpisuje tekst, ale React musi go gdzieś przechowywać. Najpierw pokażemy to tylko na jednym polu.

W App() dodaj pod innymi stanami:

```
const [adres, setAdres] = useState("");
```

Przed return dopisz funkcję:

```
function zmienAdres(event) {
    setAdres(event.target.value);
}
```

event to informacja o zdarzeniu, event.target to element, w którym się ono wydarzyło, a event.target.value to obecnie wpisany tekst.

W istniejącym polu Adres WWW dodaj dwie właściwości:

```
value={adres}
onChange={zmienAdres}
```

Na chwilę pod formularzem wpisz:

```
<p>Wpisany adres: {adres}</p>
```

SPRAWDŹ: Podczas pisania w polu Adres WWW tekst w paragrafie zmienia się na bieżąco. Potem usuń tymczasowy paragraf.

10 Odczytaj login i hasło

Powtórz dokładnie tę samą zasadę. Dodaj kolejne dwa stany:

```
const [login, setLogin] = useState("");
const [haslo, setHaslo] = useState("");
```

Dopisz dwie funkcje obsługujące zmianę tekstu:

```
function zmienLogin(event) {
    setLogin(event.target.value);
}
function zmienHaslo(event) {
    setHaslo(event.target.value);
}
```

Teraz do pola Login dodaj:

```
value={login}
onChange={zmienLogin}
```

Do pola Hasło dodaj:

```
value={haslo}
onChange={zmienHaslo}
```

W polu hasła zostaw także disabled={generujHaslo}. Dzięki temu opcja „Wygeneruj hasło” nadal blokuje możliwość wpisywania.

SPRAWDŹ: Adres i login da się wpisywać. Hasło można wpisać dopiero po odznaczeniu checkboxa.

11 Obsłuż zatwierdzenie formularza

Kliknięcie przycisku typu submit domyślnie próbuje wysłać formularz. Chcemy przejąć obsługę tego zdarzenia w React.

Przed return utwórz funkcję:

```
function zapisz(event) {
    event.preventDefault();
    alert("Zatwierdzono formularz");
}
```

Zmień otwierający znacznik formularza:

```
<form onSubmit={zapisz}>
```

event.preventDefault() zatrzymuje domyślne wysłanie formularza i przeładowanie strony. Funkcja zapisz uruchamia się po zatwierdzeniu.

SPRAWDŹ: Kliknięcie „Zapisz” pokazuje alert, a formularz nie przeładowuje strony. To na razie tylko test zdarzenia.

12 Oddziel wpisywane dane od zapisanych

Teraz ważny problem: podsumowanie powinno zmienić się dopiero po kliknięciu Zapisz, nie po każdym wpisanym znaku. Dlatego tworzymy osobny stan z danymi zatwierdzonymi.

W App() dopisz pod pozostałymi stanami:

```
const [zapisaneDane, setZapisaneDane] = useState({
    adres: "",
    login: "",
    haslo: "",
    generujHaslo: true
});
```

13 Zapisz dane z formularza w nowym stanie

Zamień testowy alert w funkcji zapisz() na wywołanie setZapisaneDane(). Cała funkcja ma teraz wyglądać tak:

```
function zapisz(event) {
    event.preventDefault();
    setZapisaneDane({
        adres: adres,
        login: login,
        haslo: haslo,
        generujHaslo: generujHaslo
    });
}
```

W zapisie adres: adres po lewej jest nazwa pola obiektu, a po prawej - wartość obecnie wpisana do formularza.

14 Dodaj podsumowanie zapisanych danych

Za zamykającym </form>, ale w głównym , wstaw dwa paragrafy:

```
<div>
    <p className="p-2 text-primary lead">
        Adres: {zapisaneDane.adres}
    </p>
    <p className="p-2">
        Login: {zapisaneDane.login}
    </p>
</div>
```

Klasy Bootstrap: p-2 to wewnętrzny odstęp 0,5 rem, text-primary to niebieski tekst, lead wyróżnia pierwszy paragraf.

> SPRAWDŹ: Wpisz adres i login, kliknij Zapisz. Potem zmień login w polu, ale NIE klikaj Zapisz - podsumowanie ma nadal pokazywać poprzedni login.

15 Wyświetl odpowiedni tekst o haśle

Ostatni paragraf zależy od tego, czy checkbox był zaznaczony w momencie zapisu. Dlatego sprawdzamy zapisaneDane.generujHaslo, a nie samą zmienną generujHaslo.

Przed return dopisz funkcję:

```
function opisHasla() {
    if (zapisaneDane.generujHaslo === true) {
        return "Hasło automatycznie generowane";
    }
    return "Hasło: " + zapisaneDane.haslo;
}
```

W bloku podsumowania, pod paragrafem z loginem, dodaj:

```
<p className="p-2">{opisHasla()}</p>
```

Test końcowy aplikacji

Przejdź przez poniższe scenariusze i przy każdym zaznacz kwadrat po lewej. Nie wpisuj prawdziwych haseł - to wyłącznie ćwiczenie.

- ☐ Na początku checkbox „Wygeneruj hasło” jest zaznaczony, a pole Hasło nieaktywne.
- ☐ Po odznaczeniu checkboxa można wpisać hasło; wpisane znaki są maskowane.
- ☐ Po zaznaczeniu checkboxa pole Hasło ponownie staje się nieaktywne.
- ☐ Po kliknięciu Zapisz pojawia się Adres i Login wpisany w formularzu.
- ☐ Z zaznaczonym checkboxem podsumowanie wyświetla: „Hasło automatycznie generowane”.
- ☐ Z odznaczonym checkboxem podsumowanie wyświetla: „Hasło: ...”.
- ☐ Po zmianie adresu lub loginu bez zatwierdzenia podsumowanie pozostaje bez zmian.
- ☐ Przycisk Zapisz jest niebieski, a pola mają klasy Bootstrapa.

Gdy coś nie działa

Brak wyglądu Bootstrap: Sprawdź, czy pakiet bootstrap jest zainstalowany i czy na początku App.jsx jest import pliku CSS.

Pole Hasło nadal jest aktywne: Sprawdź połączenie: checked={generujHaslo}, onChange={zmienGenerowanie}, disabled={generujHaslo}.

Formularz przeładowuje stronę: W znaczniku form powinno być onSubmit={zapisz}, a w funkcji zapisz - event.preventDefault().

Podsumowanie zmienia się bez Zapisz: W paragrafach używaj zapisaneDane.adres i zapisaneDane.login, a nie bezpośrednio adres i login.